

um design doc - Eyad Soliman and Karla Armoush

Architecture & Modules:

1. Um.c

a. Responsibilities:

- i. main driver, initiates program, runs fetch, decode, execute loop

b. Data Structures:

- i. None

c. Interactions:

- i. Interacts with memory interface to load program into segment 0
- ii. Interacts with the operations interface to execute instructions
- iii. Client of bitpack interface to extract fields from words

2. memory.c / memory.h

a. Responsibilities:

- i. manages segmented memory: mapping, unmapping
- ii. loads program into segment 0
- iii. implements operations for getting and putting words in segments
- iv. implements utilities for keeping track of and reusing IDs

b. Data Structures Used:

- i. Segments will be stored in a Hanson's Sequence
- ii. each segment will be represented as an array of uint32_t instructions
- iii. unused IDs will be kept track of in a Hanson Sequence

c. Interactions with other Modules:

- i. The main um.c driver will use this module to load the program into segment 0, as well as interact with the segments while performing the fetch, decode, execute loop
- ii. The operations interface will interact with this module when instructions that affect segments are executed

d. Interface Function Prototypes:

- i. Uint32_t *get_segment(uint32_t segment_ID);
- ii. Uint32_t get_word(uint32_t segment_ID, uint32_t word_offset);
- iii. Void put_word(uint32_t segment_ID, uint32_t word_offset, uint32_t word);
- iv. uint32_t map(uint32_t segment_size);
// returns ID that segment was mapped to
- v. Void unmap(uint32_t segment_ID);

3. registers.c / registers.h

a. Responsibilities:

- i. manages 8 registers

- ii. implements functions for accessing and updating registers

b. Data Structures:

- i. Registers will be contained within an array of size 8, each storing a uint32_t word

c. Interactions with other Modules:

- i. The operations interface will frequently interact with this module when performing instructions that require registers accesses
- ii. The um driver will populate the registers with 0 when the program begins

d. Interface Function Prototypes:

- i. Uint32_t get_register(uint32_t register_ID);
- ii. Void put_register(uint32_t register_ID, uint32_t value);

4. operations.c / operations.h

a. Responsibilities:

- i. Implements the 14 um RISC-style instructions

b. Data Structures:

- i. None

c. Interactions with other Modules:

- i. Will frequently interact with memory segments and registers depending on the instruction being executed.
- ii. The um driver will be a client of this module and frequently call upon it after decoding an instruction and executing it

d. Interface Function Prototypes:

- i. Void conditional_move(uint32_t ra, uint32_t rb, uint32_t rc);
- ii. Void segmented_load(uint32_t ra, uint32_t rb, uint32_t rc);
- iii. Void segmented_store(uint32_t ra, uint32_t rb, uint32_t rc);
- iv. Void addition(uint32_t ra, uint32_t rb, uint32_t rc);
- v. Void multiplication(uint32_t ra, uint32_t rb, uint32_t rc);
- vi. Void division(uint32_t ra, uint32_t rb, uint32_t rc);
- vii. Void nand(uint32_t ra, uint32_t rb, uint32_t rc);
- viii. Void halt();
- ix. Void map_segment(uint32_t rb, uint32_t rc);
- x. Void unmap_segment(uint32_t rc);
- xi. Void output(uint32_t rc);
- xii. Void input(uint32_t rc);
- xiii. Void load_program(uint32_t rb, uint32_t rc);
- xiv. Void load_value(uint32_t ra, uint32_t value);

Function Contracts for Three Most Interesting Functions:

////////////////////

name: load_program

purpose: duplicates specified segment into segment 0, updates program counter to new location

parameters:

- uint32_t rb: register ID in bitfield B (contains segment ID)
- uint32_t rc: register ID in bitfield C (contains program counter position)

```
returns: none
```

effects: updates program counter to new location, replaces segment 0 with chosen segment, if

checked errors: none

////////////////////

////////////////////

name: map_segment

purpose: creates a new segment of specified length, contains words initialized to 0

parameters:

- uint32_t rb: register ID in bitfield B (where we store its segment ID)
- uint32_t rc: register ID in bitfield C (length of segment in words)

```
returns: none
```

effects: places ID of mapped segment into register B

checked errors: none

////////////////////

//////

name: unmap_segment

purpose: frees specified segment

parameters:

- `uint32_t rc`: register containing segment's ID

```
returns: none
```

effects: calls the unmap function in the memory interface

checked errors: the unmap function called asserts that a segment exists with specified ID

////////////////////

Implementation & Testing Plan:

1. um.c driver module

- a. Implement initialization of the program (Initial state: registers are zeroed, only segment 0 is mapped, program counter points to \$m[0][0]).
 - i. Open the .um program passed on the cmd line
 - 1. **Test:** opening a file works and error handling if no file is given

2. memory.c module

- a. Implement the architecture of the module, including the sequence of segments and sequence of IDs
- b. Start creating functions for interacting with these sequences (will be needed by um.c module)
 - i. write map function which takes a segment size and initializes an segment with words containing 0
 - 1. This will require us to implement our system of tracking unused IDs
 - ii. **Test:** this works by then implementing our getters for segments and words, and test that we can put and extract words from segments that we have stored using our map_segment function
- c. We will then implement the unmapping of a segment and ensure that it properly frees the segment and underlying memory, as well as recycles the ID by appending it to our sequence of unused IDs
 - i. **Test:** map a segment and then free it and ensure that it removes the segment without memory leaks, and appends its ID to our sequence of unused IDs

3. Return to um driver

- a. Implement the loop that will read .um file content, use bitpack interface to pack words, and populate a mapped segment with those words using the memory interfaces put_word function
 - i. **Test:** Print words to ensure we are reading and packing correctly
 - ii. **Test:** test extracting words from the segments to ensure they are being put correctly

4. Implement our register interface

- a. Create our array of size 8 representing the 8 registers
- b. Implement operations for interacting with the registers
 - i. Create our put_register function that places a value into a register
 - ii. Create our get_register function that reads a value from a register

- iii. **Test:** trivial test of putting something in the register and then extracting it and ensuring it worked

5. Implement our RISC operations

- a. Start with implementing the halt and load_value instructions
- b. Implement a wrapper function that contains a switch statement, this function is called by the um driver, it is passed an opcode and determines which instruction to execute
 - i. **Test:** given a specific opcode it jumps to the right case
- c. Implement each simple operation
 - i. **Test:** each operation with random inputs and ensure correct outputs

6. Return to um driver

- a. Implement or fetch, decode, execute loop
- b. Initialize our program counter to the first word in segment 0
 - i. **Test:** our um driver with programs that have different instructions, this is the 2nd aspect of testing which is described below:

7. **Test:** This will be the stage of the program where we begin the **2nd aspect of testing**, which is that our program correctly executes the instructions given by a .um file

- a. We will use the umlab program to create .um files which specific sets of instructions, these instructions could have associated input or outputs. Then we will run our program on those .um files, comparing the inputs and outputs and ensuring that our program executes as intended.
 - i. First we will test that the halt and load_value instructions work as these are two emphasized and instructions that we should ensure are executed correctly
 - 1. We will use umlab to create a .um file which loads values into registers, then extracts them and outputs those register values, terminating with a halt at the end
 - ii. Then we will begin testing using load_program to perform a conditional jump within the program, this will be done by loading 0 into register B, and a certain position to jump to in register C, and then executing the load_program function.
- b. We will then test that segment related functions work. We will create a .um program using umlab that performs segmented loads and stores. Then within these segments will be words that have specific instructions, such as loading values into registers, and then outputting those values.
 - i. We will compare our output results with the intended outputs and ensure they are the same.

- ii. These can also have various arithmetic operations like the addition and multiplication of various values in registers, also outputting those values and ensuring they match the expected outputs.
8. Edge Cases:
- a. Test providing a program counter position that is out of the bounds of segment 0 (should unchecked runtime error)
 - b. Test trying to perform a segmented load or store on an unmapped segment (should unchecked runtime error)
 - c. Test trying to unmap segment 0 or a segment that is not mapped (should unchecked runtime error)
 - d. Test program_counter being equal to the length of the segment (reads last instruction)
 - e. Test that a load_program where $r[B] = 0$ doesn't replace segment 0 but just jumps the PC counter
 - f. Test that load_program that jumps PC counter to either word 0, or the last word still works
 - g. Test input and output with a wide range of ascii characters (0-255)
 - h. Test that you can map a segment of length 0 (still valid)
 - i. Test segmented loads and stores on the bounds of a segment (0 and segment_length - 1)

loop invariant:

Our primary loop invariant is that of our fetch, decode, execute loop.

1. Ensure that initially, the program is loaded into segment 0
2. Ensures the program counter points to the first word within that segment initially, and for every iteration of the loop, the program counter points to the next instruction to be executed
3. Upon beginning the loop, the registers are loaded with 0 values
4. In every iteration of the loop, each currently mapped segment has one unique identifier and no two segments can refer to the same array of instructions